

# Software Requirements Specification and Design Implementation (SRS)

## Bank Licensing & Compliance Portal

**Version:** 1.0

**Date:** May 10, 2026

**Prepared By:** XXX

| Version | Date       | Author | Description         |
|---------|------------|--------|---------------------|
| 1.0     | 2026-05-10 | XXX    | Technical challenge |

## 1. Introduction

### 1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements for the Bank Licensing & Compliance Portal. The portal will automate the licensing and regulatory review process used by the National Bank of Rwanda (NBR) to approve commercial banks and other financial institutions. This document serves as a reference for implementation of this technical challenges and it can be used for stakeholders, developers, testers, and reviewers to understand the system scope, architecture, business rules, and technical requirements.

### 1.2 Scope

The bank Licensing & Compliance Portal is a secure web-based application that enables:

- Applicants to submit licensing applications and supporting documents.
- Regulatory officers to review applications.
- Compliance officers to perform regulatory checks.
- Approvers to make final approval or rejection decisions.
- Administrators to manage users and reference data.
- The system to maintain an immutable audit trail of all actions.

### 1.3 References

- IEEE 830 Software Requirements Specification

## 2. Overall Description

### 2.1 Product perspective

The system is a new standalone application composed of:

- Backend REST API
- Relational database
- File Storage
- Web frontend (UI)

### 2.2 Product Functions

Key function to include:

- User Registration
- User Authentication (Login)
- Application submission
- Document upload and versioning
- Workflow processing (review, approval, reject ...)
- Audit Logging
- Notifications (options)

### 2.3 User classes and Characteristics

| Role               | Description                                            |
|--------------------|--------------------------------------------------------|
| Applicant          | Commercial bank / Financial institution representative |
| Reviewer           | Review initial including completeness and business     |
| Compliance officer | Perform regulatory compliance verification             |
| Approver           | Make final approval or rejection                       |
| Admin              | Manage users and system configurations                 |

### 2.4 Tech stacks (Operating Environment).

#### Backend

- Java 17
- Spring Boot 4.0.6
- Spring Security
- PostgreSQL
- Flyway

## **Frontend**

- React
- TypeScript
- Vite

## **Deployment**

- Deploy it as jar embedded with tomcat or
- Docker and Docker compose

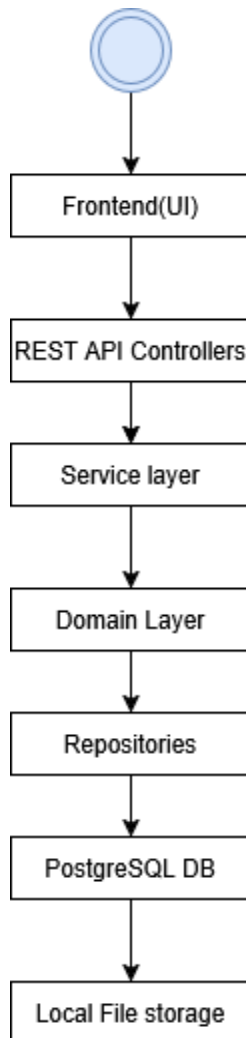
## **2.5 Design Constraints**

- Maximum upload size: 5MB per file.
- Final decisions are immutable.
- Audit log is append-only
- Review and approver must be different users.
- System must check if concurrent users are not accessing the same application.
- Unauthorized access returns HTTP 403

### 3. System Architecture

#### 3.1 Architecture Overview

The solution follows a layered architecture.



### 3.2 Reason for Layered Architecture and Principles.

As financial system, I will propose this architecture over the rest because of the following reasons:

- REST APIs provide a uniform interface, enabling integration with external systems without altering internal layers.
- Separation of concerns
- Stateless backend
- Transactional consistency
- Defense in depth
- Auditability by design.

## 4. User Roles and Permissions

### 4.1 Permission Matrix

| Action                       | Applicant | Reviewer | Compliance | Approver | Admin |
|------------------------------|-----------|----------|------------|----------|-------|
| Submit application           | ✓         |          |            |          |       |
| Edit draft                   | ✓         |          |            |          |       |
| Upload documents             | ✓         |          |            |          |       |
| View on applications         | ✓         |          |            |          |       |
| Review submitted application |           | ✓        |            |          |       |
| Requested additional info    |           | ✓        | ✓          |          |       |
| Compliance check             |           |          | ✓          |          |       |
| Approval/Reject              |           |          |            | ✓        |       |
| Manage users &config         |           |          |            |          | ✓     |
| View Audits logs             |           | ✓        | ✓          | ✓        | ✓     |

### 4.2 Segregation of Duties Rule

The user who reviewed an application cannot be the same user who performs the final approval.

Pseudocode => if reviewedBy == approvedBy: reject action

## 5. Workflow State Machine.

### 5.1 States

- Draft
- Submitted
- Under\_Review
- Information\_Requested
- Resubmitted
- Compliance\_check
- Approved
- Rejected

### 5.2 State Transition Diagram

| Initial State         | Action                         | Next State            | Notes                                  |
|-----------------------|--------------------------------|-----------------------|----------------------------------------|
| START                 | Create Draft                   | DRAFT                 | Applicant start a new application      |
| DRAFT                 | Save Draft                     | DRAFT                 | Draft remains editable                 |
| DRAFT                 | submit                         | SUBMITTED             | Applicant is formally submitted        |
| SUBMITTED             | Assign for Review              | UNDER_REVIEW          | Reviewer begins assessment             |
| UNDER_REVIEW          | Request Additional Information | INFORMATION_REQUESTED | Applicant must provide missing data    |
| UNDER_REVIEW          | Complete Review                | COMPLIANCE_CHECK      | Sent to compliance officer             |
| INFORMATION_REQUESTED | Resubmit                       | RESUBMITTED           | Applicant submits updated information  |
| RESUBMITTED           | Reopen review                  | UNDER_REVIEW          | Reviewer reassesses updated submission |

|                  |                                      |                       |                                                    |
|------------------|--------------------------------------|-----------------------|----------------------------------------------------|
| COMPLIANCE_CHECK | Request<br>Additional<br>Information | INFORMATION_REQUESTED | Compliance<br>officer<br>needs more<br>information |
| COMPLIANCE_CHECK | Approval                             | APPROVED              | Final<br>decision<br>(terminal<br>state)           |
| COMPLIANCE_CHECK | Reject                               | REJECTED              | Final<br>decision<br>(terminal<br>state)           |

### Terminal States

Once an application reaches either of these states, no further transition is allowed:

- **Approved**
- **Rejected**

**Note:** The backend validates all transitions before persisting changes.

## 6. Functional Requirements

### 6.1 Definition

#### User Management

- **FR-001 Register as User:** User/applicant shall register their account.
- **FR-002 User Approved:** Admin shall approve and assign roles to a registered user
- **FR-003 Create User:** Administrators shall create users and assign roles.
- **FR-004 Create User:** Administrators shall disable user account.

#### Authentication

- **FR-010 Login:** Users shall authenticate using username/email and password.
- **FR-011 Token Issuance:** Upon successful authentication, the system shall issue a JWT.
- **FR-012 Token Validation:** All protected endpoints shall require a valid JWT.

#### Application Management

- **FR-020 Create Draft:** Applicants shall create applications in draft state
- **FR-021 Submit Application:** Applicants shall submit completed drafts.
- **FR-022 View status:** Applicants shall view real-time application status.

#### Document Management

- **FR-030 Upload document:** Applicants shall upload supporting files up to 5MB.
- **FR-030 Store Metadata:** The system shall store file metadata.
- **FR-031 Version Documents:** Resubmitted documents shall create new versions.

#### **Review Process**

- **FR-040 Start review:** Reviewers shall move applications to UNDER\_REVIEW.
- **FR-041 Request Information:** Reviewers and compliance officers may request additional information.

#### **Compliance Check**

- **FR-050 Compliance Verification:** Compliance officers shall record compliance findings.

#### **Approval Process**

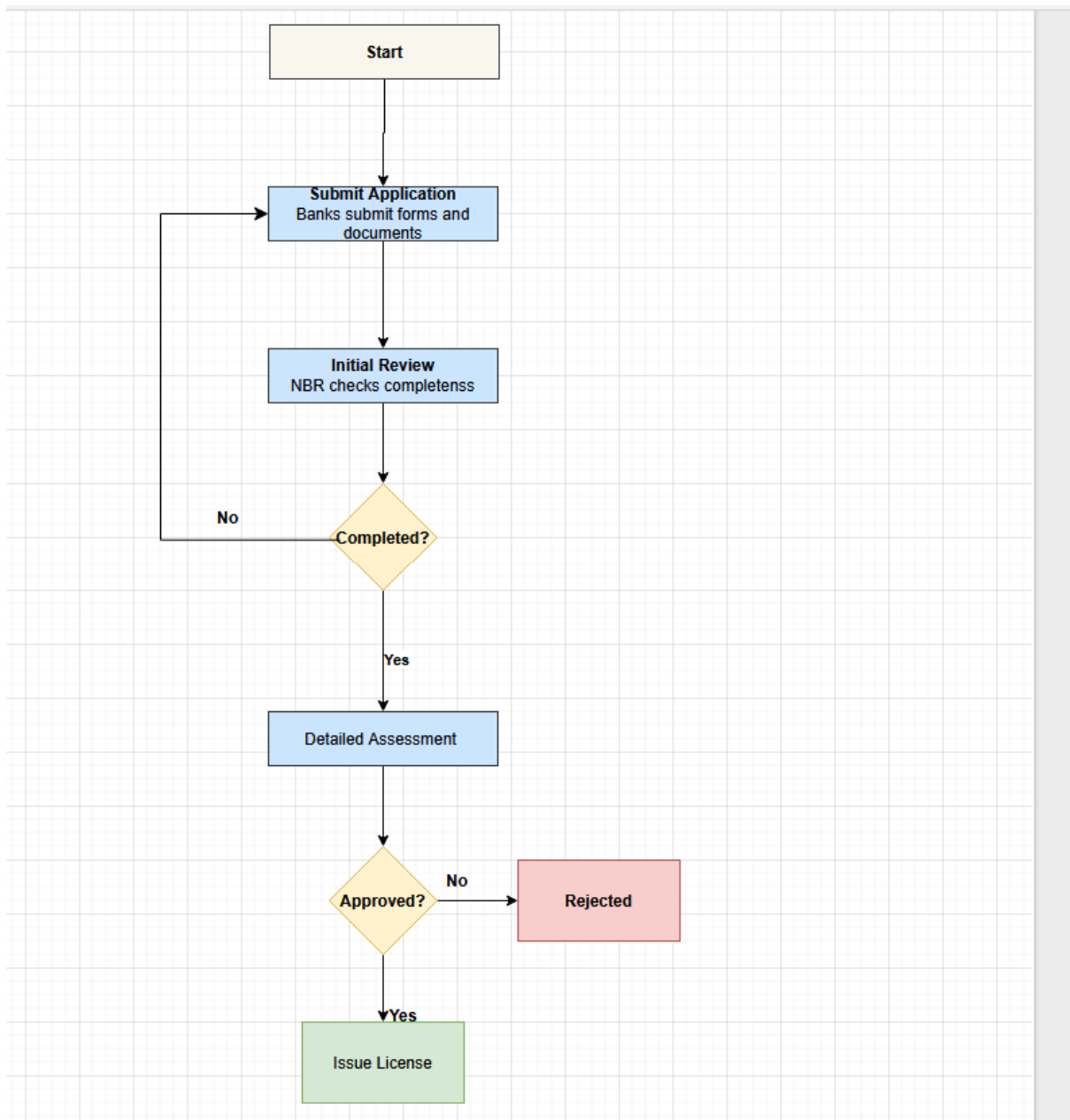
- **FR-060 Final Decision** Approvers shall approve or reject applications.
- **FR-061 Enforce SoD:** Approvers shall be different from reviewers.

#### **Audit Logging**

- **FR-070 Log Actions:** Every action shall create an audit record.
- **FR-071 Append-Only:** Audit entries shall never be updated or deleted.



## 6.2 PROCESS FLOW DIAGRAM



## 7. Non-Functional Requirements

### Security

- Password hashed using Bcrypt
- HTTPS required in production.
- Role checks enforced server-side

### Performance

- API response time under 2 seconds for normal operations.

- ACID Complaints transaction
- Immutable records
- Modular architecture
- Automated tests
- Stateless backend for horizontal scaling
- Clear role-based navigation

## 8. Data Requirement (Database Design)

### 8.1 Data Dictionary

#### 8.1.1 User

| Field         | Type      |
|---------------|-----------|
| id            | UUID      |
| username      | Varchar   |
| Email         | Varchar   |
| Password_hash | Varchar   |
| Role          | Enum      |
| Active        | Boolean   |
| Created_at    | timestamp |

#### 8.1.2 Application

| Field             | Type      |
|-------------------|-----------|
| id                | UUID      |
| reference_number  | Varchar   |
| applicant_id      | UUID      |
| current_state     | Enum      |
| reviewed_by       | UUID      |
| approved_by       | UUID      |
| submitted_at      | timestamp |
| final_decision_at | Timestamp |
| version           | integer   |

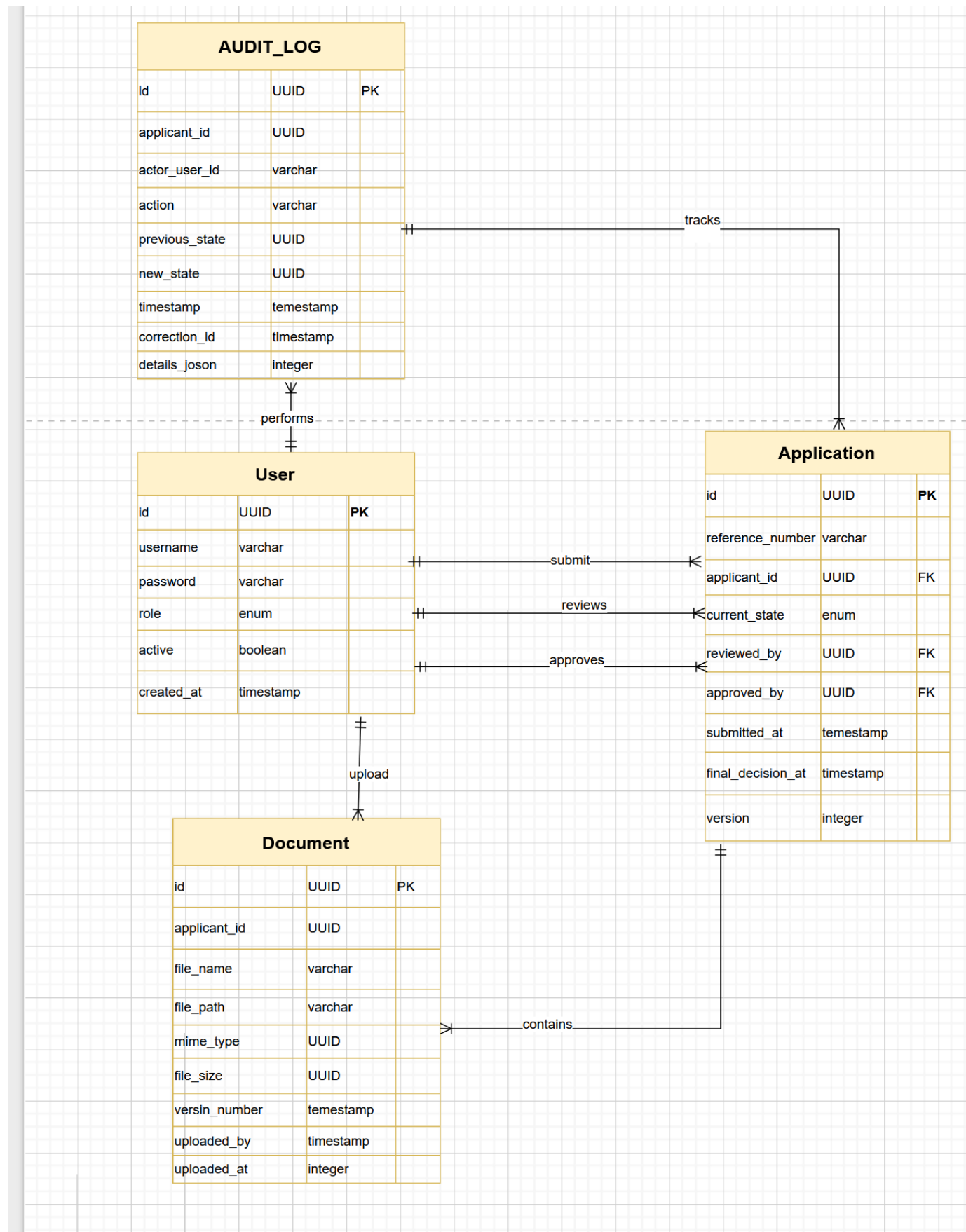
#### 8.1.3 Document

| Field          | Type      |
|----------------|-----------|
| id             | UUID      |
| applicant_id   | UUID      |
| file_name      | Varchar   |
| file_path      | Varchar   |
| mime_type      | Varchar   |
| file_size      | bigint    |
| Version_number | Integer   |
| uploaded_by    | UUID      |
| uploaded_at    | timestamp |

#### 8.1.4 Audit Log

| Field          | Type      |
|----------------|-----------|
| id             | UUID      |
| applicant_id   | UUID      |
| actor_user_id  | UUID      |
| action         | Varchar   |
| previous_state | Varchar   |
| new_state      | Varchar   |
| timestamp      | Timestamp |
| correction_id  | UUID      |
| details_json   | jsonb     |

## 8.2 Entity Relationship Diagram



## 9. Audit trail Design

### 9.1 Requirement

- Record all actions
- Be append only
- Capture before and after state
- Keep legal evidence
- Application code never updates audits records
- Database role used by application has INSERT and SELECT only.
- No UPDATE OR DELETE privileges on audit log
- Option database trigger blocks modification

## 10. Concurrency Control

### 10.1 Requirement

- Two users may act on the same application simultaneously, the solution is Optimistic **locking** using a **version** column.
- User reads application with version number.
- Update includes expected version
- Database updates only if version matches
- Otherwise, return HTTP 409 Conflict.

## 11. API Requirements

### 11.1 REST Principles

- Resource-oriented URLs.
- Standards HTTP methods
- JSON request/response.

### 11.2 Endpoints Example

#### Applications

- GET /api/applications
- POST /api/applications
- GET /api/applications/{id}
- POST /api/applications/{id}/submit
- POST /api/applications/{id}/request-info
- POST /api/applications/{id}/approve
- POST /api/applications/{id}/reject

## **Documents**

- POST /api/applications/{id}/documents

## **Audit Logs**

- GET /api/applications/{id}/audit

## **12. Trade-Offs and Design Decisions**

### **JWT vs Authentication**

JWT was selected because:

- Backend remains stateless
- Easier scaling
- Simplified front integration

#### **Trade -off:**

- Token revocation is more complex.

### **Local File Storage vs Cloud Storage**

Local storage is sufficient for this assessment.

#### **Trade -off:**

- Not suitable for multi0server production environments

### **Optimistic Locking**

Chosen because conflicts are expected to be infrequent

#### **Trade -off:**

- Users may need to retry.